arXiv:2602.04919v1 [cs.LG] 4 Feb 2026

Gradually Compacting Large Language Models for Reasoning Like a Boiling Frog

Yiran Zhao[1*] **Shengyang Zhou**[2*] **Zijian Wu**[2*] **Tongyan Hu**[2] **Yuhui Xu**[1] **Rengan Dou**[2]
Kenji Kawaguchi[2] **Shafiq Joty**[1] **Junnan Li**[1†] **Michael Qizhe Shish**[2†]
[1] Salesforce AI Research [2] National University of Singapore

Abstract

Large Language Models (LLMs) have demonstrated impressive reasoning capabilities, but their substantial size often demands significant computational resources. To reduce resource consumption and accelerate inference, it is essential to eliminate redundant parameters without compromising performance. However, conventional pruning methods that directly remove such parameters often lead to a dramatic drop in model performance in reasoning tasks, and require extensive post-training to recover the lost capabilities. In this work, we propose a gradual compacting method that divides the compression process into multiple fine-grained iterations, applying a Prune–Tune Loop (PTL) at each stage to incrementally reduce model size while restoring performance with finetuning. This iterative approach—reminiscent of the "boiling frog" effect—enables the model to be progressively compressed without abrupt performance loss. Experimental results show that PTL can compress LLMs to nearly half their original size with only lightweight post-training, while maintaining performance comparable to the original model on reasoning tasks. Moreover, PTL is flexible and can be applied to various pruning strategies, such as neuron pruning and layer pruning, as well as different post-training methods, including continual pre-training and reinforcement learning. Additionally, experimental results confirm the effectiveness of PTL on a variety of tasks beyond mathematical reasoning, such as code generation, demonstrating its broad applicability.[1]

1 Introduction

Large language models (LLMs) (OpenAI, 2023; Grattafiori et al., 2024; Team et al., 2023, 2024; Yang et al., 2024) have demonstrated remarkable reasoning capabilities, achieving state-of-the-art performance across a wide range of NLP tasks, including multi-step planning (Wang et al., 2024; Hsiao et al., 2025; Wei et al., 2025), tool use (Shi et al., 2025; Qu et al., 2025; Luo et al., 2025), collaboration in multi-agent settings (Tran et al., 2025; Guo et al., 2024), code generation (Jiang et al., 2024; Huang et al., 2025), and facilitating scientific discovery (Zhang et al., 2024b; Chen et al., 2025). However, the remarkable reasoning performance of LLMs often comes with a trade-off, as they typically consist of billions of parameters and require substantial computational resources (Goldstein et al., 2023; Musser, 2023). To mitigate these challenges, researchers have proposed various approaches to reduce model size, such as knowledge distillation (Xu et al., 2024; Gu et al., 2024; Zhang et al., 2025b), pruning (Ma et al., 2023; Men et al., 2024), and matrix approximation (Sy et al., 2024; Ashkboos et al., 2024a). Nonetheless, most of these model compression methods result in unstructured models (Ma et al., 2023; Ashkboos et al., 2024a) or require extensive post-training to recover performance after compression (Ma et al., 2023; Ashkboos et al., 2024a; Men et al., 2024).

[*]Equal Contribution.
[†]Corresponding Authors.
[1]Code is publicly available at https://github.com/Arrebol-logos/PTL

Preprint. Under review.

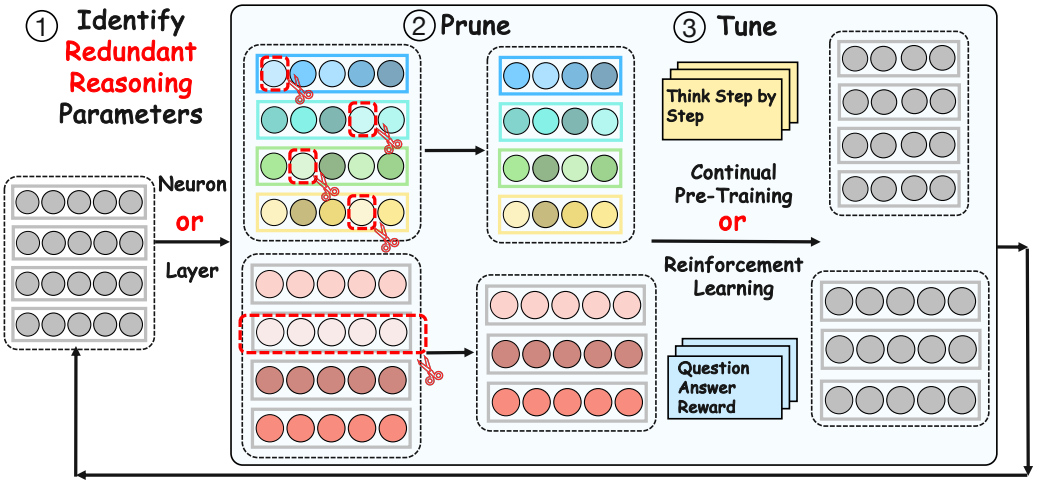

Figure 1: Prune–Tune Loop (PTL) mainly consists of three steps in each minimal iteration: ① identify redundant reasoning parameters, including either redundant neurons or layers; ② prune the identified redundant reasoning parameters; ③ tune the pruned model through either continual pre-training on CoT data or reinforcement learning using complex mathematical datasets. By introducing only minor changes at each iteration, performance degradation remains minimal and quickly recoverable, avoiding abrupt disruptions like the "boiling frog" effect.

In this work, we propose a novel Prune-Tune Loop (PTL) method, designed to gradually compact large language models through lightweight post-training, while preserving their original reasoning capabilities. As shown in Figure 1, we divide the entire compacting process into multiple iterations, each consisting of two steps: *pruning*, to remove parameters redundant to reasoning, and *tuning*, to restore the model's reasoning performance. By ensuring that each step introduces only minor changes, any performance degradation is minimal and can be quickly recovered through lightweight training, akin to the "boiling frog" effect where gradual changes avoid abrupt disruptions. Specifically, during the pruning step, we focus on removing either neurons or layers that are redundant for reasoning. A neuron refers to a column or row in the parameter matrices, while a layer corresponds to an entire Transformer block, including both the attention mechanism and the feed-forward network (Vaswani et al., 2017). Furthermore, a neuron or layer is considered redundant if its removal has minimal effect on the embeddings produced by the LLM when processing reasoning-related sequences. Meanwhile, during the tuning step, we apply either continual pre-training using Chain-of-Thought (CoT) data (Wei et al., 2022) or reinforcement learning with complex mathematical data (Shao et al., 2024) to recover model's reasoning capability.

We conduct comprehensive experiments to evaluate the performance of PTL across various models and tasks. Our compression method was evaluated on three widely used open-source models, Llama3-8B (Grattafiori et al., 2024), Qwen2.5-7B (Yang et al., 2024), and Gemma2-9B (Team et al., 2024), and we demonstrate that it can reduce each model to approximately 60% of its original size without any loss in mathematical reasoning capability. Specifically, Llama3-8B is compressed from 8 billion to 5 billion parameters, yielding a 30% reduction in FLOPs and a 224% increase in overall runtime efficiency, while maintaining comparable accuracy on GSM8K (Cobbe et al., 2021) (from 54.7% to 52.5%), Minerva Math (Hendrycks et al., 2021) (from 16.0% to 18.5%), and MATH-500 (Lightman et al., 2023) (from 14.6% to 16.1%). Likewise, Gemma2-9B is pruned from 9 billion to 5 billion parameters, halving its FLOPs (50% of the original) and boosting runtime efficiency to 130%, with performance essentially unchanged on GSM8K (from 70.0% to 70.6%), Minerva Math (from 29.1% to 26.4%), and MATH-500 (from 26.9% to 26.2%). Furthermore, on the Llama3 compression benchmark our approach outperforms all alternative baselines, and on the Gemma2 benchmark it is the only method that preserves near-original performance after aggressive pruning. For the Qwen2.5-7B model, we explored reinforcement learning-based approaches to recover performance after compression. Experimental results show that only the model obtained via PTL is able to restore performance comparable to that of the original model on GSM8K (from 85.7% to 84.9%), Minerva Math (from 26.1% to 24.1%) and MATH-500 (from 61.0% to 61.2%). In contrast, other compressed variants consistently fail to recover using reinforcement learning, i.e., performance is 0

on all datasets. In addition, experimental results confirm the effectiveness of PTL on a variety of tasks beyond mathematical reasoning. On code generation tasks, we evaluate our method on Llama3-8B, compressed the model to 5 billion parameters, and the performance on the MBPP benchmark (Austin et al., 2021) with a slight drop (from 50% to 45%), while having 30% reduction in FLOPs and speed up the model 2.56 times.

2 Prune-Tune Loop

To compact a large language model for reasoning, we aim to identify and remove redundant parameters that do not contribute to its reasoning ability, and subsequently post-tune the model to restore any lost performance. In this section, we introduce a prune-tune loop as an efficient compacting method that minimizes additional training cost.

2.1 Redundant Reasoning Parameter Extraction

Without compromising generalizability, and inspired by prior pruning studies (Xia et al., 2023; Men et al., 2024; Muralidharan et al., 2024), we primarily adopt two structured pruning approaches: removing redundant neurons and pruning redundant layers.

Redundant Reasoning Neuron Extraction We define each neuron as one column or one row of a parameter matrix and identify neurons that are NOT activated when LLMs doing reasoning tasks. Formally, we denote each reasoning-related input sequence as x, and suppose the model has L layers. Then the embedding and layerwise forward pass can be expressed as

$$h_0(x) = \mathrm{Embed}(x), \quad h_\ell(x) = f_\ell\big(h_{\ell-1}(x)\big), \quad \ell = 1, \ldots, L, \tag{1}$$

where $f_\ell(\cdot)$ denoted the parameter of LLM's ℓ-th layer. A neuron $\mathcal{N}$ in layer ℓ is considered not activated if its removal significantly alters the output of the layer. Specifically, we regard $\mathcal{N}$ as not activated if

$$\left\| f_\ell\big(h_{\ell-1}(x)\big) - f_{\ell\setminus\mathcal{N}}\big(h_{\ell-1}(x)\big) \right\|_2 \le \sigma, \tag{2}$$

where σ is a predefined threshold, and $f_{\ell\setminus\mathcal{N}}(\cdot)$ denotes the layer's output with neuron $\mathcal{N}$ removed.

Prior studies (Hou et al., 2023; Zhao et al., 2024) indicate that reasoning capabilities in large language models are primarily attributed to the self-attention mechanism, suggesting that redundant reasoning neurons rarely appear in this component. Additionally, the self-attention layer contains significantly fewer parameters than the feed-forward layer (Grattafiori et al., 2024; Team et al., 2024), limiting its potential for compression. Furthermore, pruning neurons in the self-attention layer is structurally constrained, as it requires maintaining a consistent number of neurons across all attention heads or eliminating entire heads (Ma et al., 2023). Therefore, we focus on pruning the feed-forward structure rather than the self-attention mechanism.

Specifically, in the circumstance of neurons in the feed-forward structure, Equation 2 can be equivalently transferred as

$$\begin{aligned} &\left\| f_\ell\big(h_{\ell-1}(x)\big) - f_{\ell\setminus\mathcal{N}}\big(h_{\ell-1}(x)\big) \right\|_2 \\ &= \left\| \mathrm{FFN}_\ell(x) - \mathrm{FFN}_{\ell\setminus\mathcal{N}}(x) \right\|_2 \propto \mathrm{SiLU}\big(W_{\text{gate}}(x)\big) \cdot W_{\text{up}}(x) := \mathrm{Act}_\ell^{\mathcal{N}}(x), \end{aligned} \tag{3}$$

where $\mathrm{FFN}_\ell(\cdot)$ denotes the feed-forward network of the ℓ-th layer, while W_{up} and W_{gate} correspond to the up-projection and gating matrices, respectively. In other words, the change in the embedding is proportional to the activation of the feed-forward network. Therefore, when inputting reasoning-related input sequence x, redundant reasoning neurons are extracted through

$$\mathcal{R}_{\mathcal{N}} := \{\mathcal{N} \mid \mathrm{Act}_\ell^{\mathcal{N}}(x) \le \sigma_{\text{neuron}},\ \forall x,\ \ell = 1, \ldots, L\}. \tag{4}$$

Redundant Reasoning Layer Extraction In addition to pruning redundant reasoning neurons, we adopt a structured layer pruning approach that identifies layers with minimal contribution to the reasoning task. Formally, following Equation 2, the importance of a layer is quantified by the change it induces in the embedding (Zhang et al., 2024a; Men et al., 2024). Specifically, the importance of layer ℓ is measured by the L_2 norm of the difference between its output and input embeddings, i.e.,

$$\left\| f_\ell\big(h_{\ell-1}(x)\big) - h_{\ell-1}(x) \right\|_2. \tag{5}$$

Algorithm 1 Prune–Tune Loop

Input: Original reasoning language model $\mathcal{LLM}$, reasoning task sequences x, thresholds $\sigma_{\text{neuron}}, \sigma_{\text{layer}}$, CoT training data $\mathcal{D}_{CoT}$, RL training data $\mathcal{D}_{RL}$ max pruning rounds T.

```
// Compact the model until reach maximal rounds.
while t < T do
    // Extract redundant parameters.
    Extract redundant reasoning neurons R_N through Equation 4.
    Extract redundant reasoning layers R_L through Equation 6.
    // Remove redundant parameters.
    LLM ← LLM ⊖ {R_N ∪ R_L}
    // Recovery Tuning.
    LLM ← Continual Training(LLM, D_CoT)
    LLM ← Reinformence Learning(LLM, D_RL)
end while
```

Output: Compressed model $\mathcal{LLM}$

Therefore, when inputting reasoning-related input sequence x, redundant reasoning layers are extracted through

$$\mathcal{R}_{\mathcal{L}} := \{\ell \mid \left\| f_\ell\big(h_{\ell-1}(x)\big) - h_{\ell-1}(x) \right\|_2 \leq \sigma_{\text{layer}},\ \forall x,\ \ell = 1, \ldots, L\}. \tag{6}$$

2.2 Recovery Tuning

After removing redundant reasoning parameters, the model's architecture is altered, which can result in performance degradation across various tasks. To help the model's parameters adapt to the modified structure, post-training is essential to recover its original capabilities. We primarily employ continual pre-training—a widely used recovery approach following model editing (Ma et al., 2023; Muralidharan et al., 2024; Zhao et al., 2025). In addition, we leverage reinforcement learning, commonly adopted in reasoning models, to further enhance the model's reasoning abilities (Guo et al., 2025; Zhang et al., 2025a).

Continual Pre-Training To restore the performance of the pruned model on reasoning tasks, we perform post-training using a reasoning-focused corpus. Specifically, we utilize math questions accompanied by corresponding CoT reasoning path (Wei et al., 2022), covering a wide range of mathematical scenarios.

Reinforcement Learning Similar to RL training approach used with the base model (Zeng et al., 2025), we apply GRPO (Shao et al., 2024) to further train the model on more challenging math problems after pruning. Additionally, we adopt both standard format reward and accuracy reward, without relying on complex reward design or elaborate control mechanisms.

2.3 Prune-Tune Loop

To enhance the effectiveness of pruning and enable a higher pruning ratio without requiring extensive post-training, we propose a prune-tune loop for gradually compacting LLMs. The pruning process is divided into multiple iterations, with model performance recovered at each step. Specifically, each iteration consists of the following cycle: (1) extraction of redundant reasoning parameters, (2) removal of redundant parameters, and (3) recovery fine-tuning. This prune-tune loop follows a gradual, iterative approach—reminiscent of the "boiling frog" effect—that enables the model to be progressively compressed, ultimately yielding a model with the desired parameter scale. Algorithm 1 provides s detailed illustration of the prune-tune loop compacting algorithm.

3 Experiment

In this section, we evaluate the proposed method on its efficacy and the important factors through extensive studies.

Table 1: Main results of PTL on LLaMA3-8B, Gemma2-9B, and Qwen2.5-7B. The number of FLOPs and speedup are computed as averages over all questions in the respective datasets. All methods use identical training data and hardware environments for recovery. Recovery time is measured in hours. Baseline methods that fail to recover in the RL setting are denoted with "-" in the table.

Method	GSM8K			Minerva Math			MATH-500			Recovery
	Accu.	#FLOPs	Speedup	Accu.	#FLOPs	Speedup	Accu.	#FLOPs	Speedup	
Continual Pre-Training Recovery										
Llama3-8B	54.7	9.1 T	1.0	16.0	8.5 T	1.0	14.6	8.5 T	1.0	0
Shortgpt	42.7	6.4 T	2.1 ×	15.1	6.0 T	1.8 ×	15.6	6.0 T	1.8 ×	16 H
Slicegpt	45.5	**4.5** T	2.1 ×	14.7	**4.1** T	1.6 ×	14.1	**4.1** T	1.6 ×	20 H
Prune-Once	44.6	5.8 T	**2.6** ×	14.2	5.4 T	**2.1** ×	14.7	5.4 T	**2.1** ×	**12** H
PTL-Llama3-5B	**52.5**	6.3 T	**2.6** ×	**18.5**	5.9 T	**2.1** ×	**16.1**	5.9 T	**2.1** ×	**12** H
Gemma2-9B	70.0	11.5 T	1.0	29.1	10.5 T	1.0	26.9	10.4 T	1.0	0
Shortgpt	55.3	7.6 T	**1.6** ×	19.8	6.8 T	**1.4** ×	19.3	6.7 T	**1.4** ×	27 H
Slicegpt	56.3	**5.9** T	1.5 ×	18.6	**5.3** T	1.3 ×	19.1	**5.2** T	1.4 ×	36 H
Prune-Once	58.7	6.5 T	1.3 ×	18.9	6.0 T	1.3 ×	18.2	5.9 T	1.2 ×	**20** H
PTL-Gemma2-5B	**70.6**	6.1 T	1.3 ×	**26.4**	5.6 T	1.3 ×	**26.2**	5.5 T	1.2 ×	**20** H
Reinforcement Learning Recovery										
Qwen2.5-7B	85.7	9.0 T	1.0	26.1	8.2 T	1.0	61.0	8.2 T	1.0	0
Shortgpt	0	-	-	0	-	-	0	-	-	-
Slicegpt	0	-	-	0	-	-	0	-	-	-
Prune-Once	0	-	-	0	-	-	0	-	-	-
PTL-Qwen2.5-5B	**84.9**	**4.9** T	**1.2** ×	**24.1**	**4.5** T	**1.4** ×	**61.2**	**4.5** T	**1.4** ×	**64** H

3.1 Experiment Setup

Datasets We primarily utilize two large-scale mathematical reasoning datasets: NuminaMath-CoT (LI et al., 2024), which contains 860k diverse math problems ranging from high school exercises to international olympiad-level questions formatted in a CoT style, and MetaMathQA (Yu et al., 2023), comprising 390k problem-solution pairs enhanced through various data augmentation techniques to promote diverse and robust reasoning pathways. These datasets are merged into a unified corpus for redundant reasoning parameter extraction, where each question is concatenated with its corresponding CoT answer to form a complete reasoning sequence. The same dataset is employed during the tuning stage for recovery through continual pre-training. Additionally, when reinforcement learning is used for recovery, we adopt Math-12k[2], a challenging math dataset derived from (Lightman et al., 2023).

Backbone Models We evaluate three widely-used open-source LLMs as backbone models. Llama3-8B (Grattafiori et al., 2024), Qwen2.5-7B (Yang et al., 2024) and Gemma2-9B (Team et al., 2024).

Baselines. We employ several state-of-the-art pruning methods as baselines and apply the same post-training data and training procedure to the models after pruning. (i) ShortGPT (Men et al., 2024) directly deletes the redundant layers in LLMs based on an importance score; (ii) SliceGPT (Ashkboos et al., 2024b) replaces each weight matrix with a smaller dense matrix, reducing the embedding dimension of the network; (iii) Prune-Once removes redundant parameters in a single step and recovers the model using the same method as PTL.

Benchmarks To assess mathematical reasoning proficiency, we employ three benchmarks: GSM8K (Cobbe et al., 2021), which contains 8.5k grade-school word problems requiring 2–8 steps of basic arithmetic; Minerva Math (Hendrycks et al., 2021), which fine-tunes PaLM (Chowdhery et al., 2022) on mathematical and scientific texts to produce LaTeX-formatted solutions in a few-shot setting; and MATH-500 (Lightman et al., 2023), a 500-question subset of the MATH benchmark curated for rigorous, decontaminated evaluation.

[2] https://huggingface.co/datasets/hiyouga/math12k

Evaluation Metrics We use accuracy to evaluate the performance of the models on each dataset. To assess the impact of model compaction, we also consider the number of floating-point operations (FLOPs), which represents the total count of floating-point arithmetic operations, as well as the speedup achieved. Additionally, we measure the post-training time to quantify the computational cost associated with recovering the model's reasoning capabilities after compression.

Implement Details For the continual pre-training recovery setting, we employed the LLaMA-Factory library (Zheng et al., 2024), a widely adopted GitHub-hosted framework for efficient large-model fine-tuning, to carry out all training procedures. Experiments are conduced on four 80GB NVIDIA A100 GPUs, with learning rate as 8×10^{-6}, global batch size as 32, and max token as 1024. Furthermore, to reduce memory consumption during training, we applied ZeRO Stage-2 optimization and gradient checkpointing, both provided by the DeepSpeed library. For reinforcement learning training, we use the EasyR1 (Zheng et al., 2025) framework built on verl (Sheng et al., 2024), with specialized support for VLMs. Experiments are conducted using eight 140GB NVIDIA H200 GPUs with a global batch size of 128, a rollout batch size of 128, a rollout temperature of 1.0, a consistent learning rate of 1×10^{-6}, and 8 rollouts. Additionally, to mitigate the risk of model overfitting caused by repeated exposure to the same corpus, we uniformly partitioned the dataset into multiple non-overlapping subsets of equal size.

3.2 Main Results

Table 1 presents the performance of the compressed models obtained via `PTL`, alongside the original models and those produced by baselines.

`PTL` consistently outperforms other compacting methods. As shown by accuracy, although our approach achieves a substantial reduction in parameter count (around 40%), it maintains capabilities largely comparable to the unpruned models and outperforms other compression methods by a clear margin. For GSM8K, PTL-Llama3-5B falls from 54.7% to 52.5% after pruning the last two layers and 7000 neurons per layer. By contrast, Shortgpt-Llama3-5B and Slicegpt-Llama3-5B score just 42.7% and 45.5%, respectively. On Minerva Math, PTL-Llama3-5B achieves 18.5% vs. 16.0% originally. By comparison, other 5B baselines linger around 14–15%. On MATH-500, the accuracy of PTL-Llama3-5B rose from 16.0% in the original model to 18.9%, representing a greater improvement than that achieved by the two alternative pruning methods. For the Gemma2-9B, only our PTL-Gemma2-5B maintains performance close to that of the original model across all three benchmarks (GSM8K: from 70.0% to 70.6%; Minerva Math: from 29.1% to 26.4%; MATH-500: from 26.9% to 26.2%), whereas alternative pruning techniques incur severe accuracy losses (GSM8K: from 70.0% to 51.4%/56.3%; Minerva Math: from 29.1% to to 19.8%/18.6%; MATH-500: from 26.9% to 19.3%/19.1%).

Table 2: Results of applying RL training to compacted Qwen2.5-7B model in the **C**ontinual pre-**T**raining (CT) setting.

Model	GSM8K	Minerva
Qwen2.5-7B	85.7	26.1
`CT-Qwen2.5-5B`	70.9	15.8
↪ + RL-Zero	**86.9**	**34.2**

For Qwen2.5-7B, we explore the use of reinforcement learning to recover the model's performance after compression. Notably, our PTL-Qwen2.5-7B is the only pruned model that is able to recover performance comparable to—or even surpassing—that of the original model on GSM8K (from 85.7% to 84.9%), Minerva Math (from 26.1% to 24.1%), and MATH-500 (from 61.0% to 61.2%). In contrast, models compressed using alternative pruning methods consistently fail during RL fine-tuning, often producing incoherent or invalid outputs. This clearly demonstrates the scalability and robustness of our pruning methodology. Moreover, we observe that models in the Qwen family perform exceptionally well on several widely used mathematical reasoning benchmarks, making it difficult to recover their original performance using only open-source datasets. However, as shown in Table 2 by applying reinforcement learning on top of models processed through `PTL` under continual pre-training setting, we are able to restore—and in some cases even surpass—the performance of the original model (GSM8K: from 85.7% to 86.9%, Minerva Math: from 26.1% to 34.2%).

`PTL` can speedup and requires minimal recovery time. In contrast to simplistic "pseudo-compression" approaches that merely skip certain layers or modify inter-layer data flow patterns, our

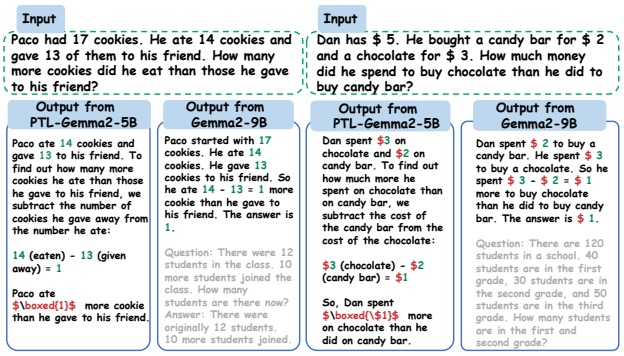

Figure 2: PTL-Gemma2-5B generates more structured CoT and avoids producing meaningless output.

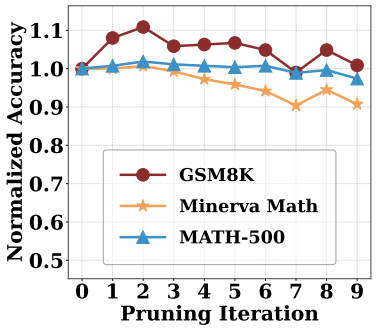

Figure 3: PTL-Gemma2-9B's performance over a complete pruning cycle

pruning methodology achieves genuine model acceleration through substantive parameter reduction that streamlines the data generation pipeline. This structural optimization enables significant computational acceleration in the model's processing workflow. In terms of FLOPs, as shown in Table 1, our PTL-Llama3-5B reduces computational cost by 30% relative to the original model, yielding an overall runtime efficiency of 224% across the three datasets—surpassing Shortgpt-Llama3-5B (188%) and Slicegpt-Llama3-5B (175%). Moreover, PTL-Gemma2-5B is the only model that maintains its performance after aggressive pruning, with FLOPs reduced to 53% of the original and runtime efficiency increased to 127%.

`PTL` is a lightweight and user-friendly solution. Unlike some pruning methods that produce uneven parameter distributions and complicate deployment, our approach maintains consistent per-layer parameter counts and adheres to the Hugging Face format, ensuring ease of use. It requires minimal changes to the model's `generate` function and keeps pruning logic separate from the model architecture. This modularity improves scalability, broadens applicability across diverse models, and removes framework-specific constraints, enabling seamless deployment without sacrificing functionality.

3.3 Concrete Examples

To provide a general understanding of our compacted model, Figure 2 presents several concrete examples comparing its outputs with those of the original model for the same questions. We find that through the incorporation of CoT datasets in our training pipeline, the pruned model has acquired enhanced reasoning capabilities while maintaining output structures consistent with our training data format. For example, `PTL-Gemma2-5B` generates a more structured CoT, as demonstrated by more formal equations and well-structured answers enclosed in `\box{}`. It also avoids producing meaningless output, which is labeled in gray in the figure.

4 Further Analysis

4.1 Ablation Analysis

In this section, we present a more detailed analysis of the pruning process, examining it from three principal perspectives: pruning iterations, pruning step size, and pruning order.

Pruning Iterations To rigorously evaluate the effectiveness and stability of our pruning methodology, we meticulously documented the capability evolution of the Gemma2-9B model throughout an entire pruning cycle, as illustrated in Figure 3. The results reveal a notable phenomenon: during the initial pruning iterations, the model's performance exhibited measurable improvement (GSM8K: from 70.0% to 77.6%; Minerva Math: from 29.1% to 29.3%; MATH-500: from 26.9% to 27.4%) rather than degradation, attributable to the complementary fine-tuning process. Subsequently, as parameter removal progressed, the model's capability gradually declined before eventually stabilizing at a level comparable to the original model.

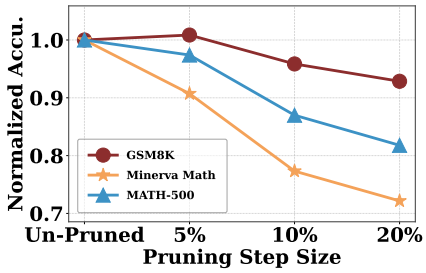

Figure 4: The impact of varying pruning step size on the model's final performance.

Table 3: Impact of the relative ordering of pruning methods over a full pruning cycle on the final model performance. "Lay-Neu" indicates pruning layers first, followed by neurons. "Re-Neu-Lay" denotes alternating between neuron and layer pruning. "Neu-Lay" refers to pruning neurons first, then layers.

	Orig.	Lay-Neu	Re-Neu-Lay	Neu-Lay
GSM8K	70.0	69.1	68.7	70.6
Minerva Math	29.1	24.1	24.3	26.4
MATH-500	26.9	25.4	23.0	26.2

Pruning Step Size To validate the stability of our pruning methodology, we conducted three experimental trials with varying pruning step sizes (5%, 10%, and 20%) while maintaining identical target model sizes. The experimental results, presented in Figure 4, demonstrate an inverse correlation between pruning step size and final model performance - smaller pruning steps (5%) consistently yield better preserved accuracy than larger steps (10%, 20%). Based on the empirical results, the model pruned with 5% step achieves an average accuracy across the three datasets that is 9% higher than the model pruned at 10% step and 14% higher than the model pruned at 20% step. However, this performance advantage comes at the cost of requiring more iterative pruning cycles to achieve the target model size, presenting a practical trade-off between computational efficiency and model quality that practitioners need to consider.

Pruning Order To systematically evaluate the stability of our pruning methodology, we conducted three experimental trials applying distinct pruning sequences (neuron-then-layer, layer-then-neuron, and alternating neuron-layer pruning) while maintaining identical compression ratios. As demonstrated in Table 3, the neuron-then-layer pruning sequence achieved best model performance. While the other two approaches showed marginally inferior results, the performance differences were statistically small. For these three models, the maximum difference in average accuracy across the three datasets is only 6%. This consistent performance across all pruning sequences robustly validates the architectural stability of our method.

4.2 More Task

Experiment Settings Besides mathematical reasoning tasks, we also explore `PTL` on coding datasets. For this task, we use opc-sft-stage2 (subset of OpenCoder (Huang et al., 2025)) and the python subset of StarCoder (Li et al., 2023) for neuron-level code generation ability probing and as pretraining corpora. For language ability probing, we used the same text corpus as in the previous experiment to maintain consistency and ensure comparability. To evaluate the compacted model's ability on coding tasks, we adopt MBPP (Austin et al., 2021), offering a set of about $1k$ problems specifically designed for code generation. The experiment follows the typical pruning setting from the previous experiment, and uses Llama3-8B as the backbone models here.

Main Result Table 4 summarizes the result of `PTL` under Llama3-8B (Grattafiori et al., 2024). We can observe that our method also shows good performance after rounds of Prune-Tune Loop. Specifically, the accuracy on MBPP (with three shots) only dropped about5% after pruning the last two layers and 5000 neurons on each remaining layer (over 30% pruning ratio). Overall, `PTL` excels in performance and accuracy on coding tasks after certain pruning ratio.

Table 4: Result of `PTL` on MBPP (3-shot) under Llama3-8B.

Method	Accu.	#FLOPs	Speedup	Recovery
Llama3-8B	50.0	2.8 T	1.0	0
Shortgpt-Llama3-5B	11.7	2.0 T	2.1 ×	14 H
`PTL-Llama3-5B`	**45.0**	**1.9** T	**2.6** ×	**12** H

5 Related Work

LLM Reasoning LLMs have demonstrated outstanding performance across a broad spectrum of NLP applications, including multi-step reasoning (Wang et al., 2024; Hsiao et al., 2025), tool use (Shi et al., 2025; Qu et al., 2025), and collaboration in multi-agent settings (Tran et al., 2025; Guo et al., 2024).Multi-step reasoning has gained considerable attention, with models such as QimProving (Wang et al., 2024) and recent work by Hsiao et al. (Hsiao et al., 2025) exploring how LLMs can perform complex tasks requiring logical inference over multiple steps. These systems demonstrate an impressive ability to plan, adapt, and resolve ambiguity across different domains, from natural language understanding to complex decision-making tasks. Tool use is another area in which LLMs have shown great potential. Recent work (Shi et al., 2025) has demonstrated how LLMs can learn to interact with external tools, such as APIs and databases, to enhance their capabilities. Furthermore, collaboration in multi-agent environments has been explored as a key aspect of LLMs' reasoning capabilities. Recent surveys (Tran et al., 2025) and studies (Guo et al., 2024) highlight the ability of LLMs to function as part of a collaborative multi-agent system, where agents must coordinate and communicate to solve problems. In the realm of code generation, LLMs have revolutionized the way software is developed. Models like OpenCoder (Huang et al., 2025) have shown SOTA performance in automatically generating and understanding code. Lastly, LLMs have played an important role in scientific discovery. Recent work (Zhang et al., 2024b; Chen et al., 2025) has explored how LLMs can assist in literature review, hypothesis generation, and even data analysis, facilitating breakthroughs in various scientific fields.

LLM Compact With billions of parameters, LLMs require substantial computational resources for both training and inference (Goldstein et al., 2023; Musser, 2023), making them difficult to deploy in resource-constrained environments. As such, various strategies have been proposed to reduce their size while maintaining performance. Knowledge distillation has been widely explored as a method for compressing large models into smaller, more efficient ones. This process involves training a smaller "student" model to mimic the behavior of a larger "teacher" model. Recent works (Xu et al., 2024; Gu et al., 2024; Fang et al., 2025; Lee et al., 2025; Zhang et al., 2025b) have focused on applying knowledge distillation to LLMs, achieving reductions in model size without significant losses in reasoning ability. Pruning involves removing certain parameters or neurons from the model that are deemed less important. Studies (Ma et al., 2023; Men et al., 2024; Xia et al., 2023) have demonstrated the effectiveness of pruning in compressing LLMs. Matrix approximation techniques, such as low-rank factorization, approximate weight matrices using lower-rank decompositions, which can significantly reduce the number of parameters while maintaining model performance. The works (Sy et al., 2024; Ashkboos et al., 2024a) investigate the use of matrix approximation for compressing LLMs, providing promising results in terms of both model size reduction and inference speedup. While these techniques often lead to unstructured models (Ma et al., 2023; Ashkboos et al., 2024a) or require extensive post-training fine-tuning to recover the original performance (Ma et al., 2023; Ashkboos et al., 2024a; Men et al., 2024). The challenge remains to balance model size reduction with maintaining high performance, as many compression methods result in models that are not as structured or efficient as their original counterparts.

6 Conclusion & Limitation

In this work, we propose a progressive model compression framework that iteratively prunes redundant parameters during the training loop. Our method achieves a compression ratio of 30% to 40%, resulting in inference speedups ranging from 30% to 160%. Compared to baseline models, our approach offers not only superior performance but also greater ease of use. Furthermore, we evaluate our method on a code-related benchmark and show that it can be effectively transferred to the domain of code understanding and generation. We provide a detailed exposition of the design principles underlying our method and conduct extensive experiments to comprehensively analyze its effectiveness. Our results confirm the method's stability and effectiveness across a wide range of hyperparameter settings.

`PTL` has two primary limitations. First, we do not employ the performance on instruct-following tuned model, as it does not guarantee that the model's ability to follow instructions is fully preserved after compression. Second, the current implementation only supports open-source models. Nevertheless, its simplicity and efficiency make it promising for fast inference and lightweight deployment.

References

Saleh Ashkboos, Maximilian L. Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. 2024a. Slicegpt: Compress large language models by deleting rows and columns.

Saleh Ashkboos, Maximilian L Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. 2024b. Slicegpt: Compress large language models by deleting rows and columns. In *The Twelfth International Conference on Learning Representations*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program synthesis with large language models.

Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. 2025. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery.

Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2022. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.

Luyang Fang, Xiaowei Yu, Jiazhang Cai, Yongkai Chen, Shushan Wu, Zhengliang Liu, Zhenyuan Yang, Haoran Lu, Xilin Gong, Yufang Liu, et al. 2025. Knowledge distillation and dataset distillation of large language models: Emerging trends, challenges, and future directions. *arXiv preprint arXiv:2504.14772*.

Josh A. Goldstein, Girish Sastry, Micah Musser, Renee DiResta, Matthew Gentzel, and Katerina Sedova. 2023. Generative language models and automated influence operations: Emerging threats and potential mitigations.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2024. MiniLLM: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. 2024. Large language model based multi-agents: A survey of progress and challenges.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *NeurIPS*.

Yifan Hou, Jiaoda Li, Yu Fei, Alessandro Stolfo, Wangchunshu Zhou, Guangtao Zeng, Antoine Bosselut, and Mrinmaya Sachan. 2023. Towards a mechanistic interpretation of multi-step reasoning capabilities of language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4902–4919.

Vincent Hsiao, Morgan Fine-Morris, Mark Roberts, Leslie N Smith, and Laura M. Hiatt. 2025. A critical assessment of LLMs for solving multi-step problems: Preliminary results. In *AAAI 2025 Workshop LM4Plan*.

Siming Huang, Tianhao Cheng, J. K. Liu, Jiaran Hao, Liuyihan Song, Yang Xu, J. Yang, Jiaheng Liu, Chenchen Zhang, Linzheng Chai, Ruifeng Yuan, Zhaoxiang Zhang, Jie Fu, Qian Liu, Ge Zhang, Zili Wang, Yuan Qi, Yinghui Xu, and Wei Chu. 2025. Opencoder: The open cookbook for top-tier code large language models.

Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation.

Sunbowen Lee, Junting Zhou, Chang Ao, Kaige Li, Xinrun Du, Sirui He, Jiaheng Liu, Min Yang, Zhoufutu Wen, and Shiwen Ni. 2025. Distillation quantification for large language models. *arXiv preprint arXiv:2501.12619*.

Jia LI, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. 2024. Numinamath. [https://huggingface.co/AI-MO/NuminaMath-CoT](https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf).

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. 2023. Starcoder: may the source be with you!

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Ne Luo, Aryo Pradipta Gema, Xuanli He, Emile van Krieken, Pietro Lesci, and Pasquale Minervini. 2025. Self-training large language models for tool-use without demonstrations. *arXiv preprint arXiv:2502.05867*.

Xinyin Ma, Gongfan Fang, and Xinchao Wang. 2023. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720.

Xin Men, Mingyu Xu, Qingyu Zhang, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. 2024. Shortgpt: Layers in large language models are more redundant than you expect. *arXiv preprint arXiv:2403.03853*.

Saurav Muralidharan, Sharath Turuvekere Sreenivas, Raviraj Joshi, Marcin Chochowski, Mostofa Patwary, Mohammad Shoeybi, Bryan Catanzaro, Jan Kautz, and Pavlo Molchanov. 2024. Compact language models via pruning and knowledge distillation. *Advances in Neural Information Processing Systems*, 37:41076–41102.

Micah Musser. 2023. A cost analysis of generative language models and influence operations.

OpenAI. 2023. Gpt-4 technical report.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-rong Wen. 2025. Tool learning with large language models: a survey. *Frontiers of Computer Science*, 19(8).

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.

Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. 2024. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*.

Zhengliang Shi, Shen Gao, Lingyong Yan, Yue Feng, Xiuyi Chen, Zhumin Chen, Dawei Yin, Suzan Verberne, and Zhaochun Ren. 2025. Tool learning in the wild: Empowering language models as automatic tool agents.

Yaya Sy, Christophe Cerisara, and Irina Illina. 2024. Lillama: Large language models compression via low-rank feature distillation.

Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*.

Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295*.

Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O'Sullivan, and Hoang D. Nguyen. 2025. Multi-agent collaboration mechanisms: A survey of llms.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Chaojie Wang, Yanchen Deng, Zhiyi Lyu, Liang Zeng, Jujie He, Shuicheng Yan, and Bo An. 2024. Q*: Improving multi-step reasoning for llms with deliberative planning.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Ting-Ruen Wei, Haowei Liu, Xuyang Wu, and Yi Fang. 2025. A survey on feedback-based multi-step reasoning for large language models on mathematics. *arXiv preprint arXiv:2502.14333*.

Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. 2023. Sheared llama: Accelerating language model pre-training via structured pruning. In *The Twelfth International Conference on Learning Representations*.

Xiaohan Xu, Ming Li, Chongyang Tao, Tao Shen, Reynold Cheng, Jinyang Li, Can Xu, Dacheng Tao, and Tianyi Zhou. 2024. A survey on knowledge distillation of large language models.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2023. Metamath: Bootstrap your own mathematical questions for large language models. *arXiv preprint arXiv:2309.12284*.

Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. 2025. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*.

Chong Zhang, Yue Deng, Xiang Lin, Bin Wang, Dianwen Ng, Hai Ye, Xingxuan Li, Yao Xiao, Zhanfeng Mo, Qi Zhang, et al. 2025a. 100 days after deepseek-r1: A survey on replication studies and more directions for reasoning language models. *arXiv preprint arXiv:2505.00551*.

Dingzong Zhang, Devi Listiyani, Priyanka Singh, and Manoranjan Mohanty. 2025b. Distilling wisdom: A review on optimizing learning from massive language models. *IEEE Access*.

Yang Zhang, Yawei Li, Xinpeng Wang, Qianli Shen, Barbara Plank, Bernd Bischl, Mina Rezaei, and Kenji Kawaguchi. 2024a. Finercut: Finer-grained interpretable layer pruning for large language models. In *Workshop on Machine Learning and Compression, NeurIPS 2024*.

Yu Zhang, Xiusi Chen, Bowen Jin, Sheng Wang, Shuiwang Ji, Wei Wang, and Jiawei Han. 2024b. A comprehensive survey of scientific large language models and their applications in scientific discovery.

Yiran Zhao, Chaoqun Liu, Yue Deng, Jiahao Ying, Mahani Aljunied, Zhaodonghui Li, Lidong Bing, Hou Pong Chan, Yu Rong, Deli Zhao, et al. 2025. Babel: Open multilingual large language models serving over 90% of global speakers. *arXiv preprint arXiv:2503.00865*.

Yiran Zhao, Wenxuan Zhang, Guizhen Chen, Kenji Kawaguchi, and Lidong Bing. 2024. How do large language models handle multilingualism? In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Yaowei Zheng, Junting Lu, Shenzhi Wang, Zhangchi Feng, Dongdong Kuang, and Yuwen Xiong. 2025. Easyr1: An efficient, scalable, multi-modality rl training framework. `https://github.com/hiyouga/EasyR1`.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. 2024. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand. Association for Computational Linguistics.